



---

Nama: **Dinda Joycehana (122140048)**  
Mata Kuliah: **Pengolahan Sinyal Digital (IF3024)**

---

Tugas: **Final Project**  
Tanggal: 31 Mei 2025

## 1 Deskripsi Proyek

Proyek ini mengimplementasikan sistem komprehensif yang menggabungkan pengukuran sinyal respirasi dan pengolahan sinyal remote photoplethysmography (rPPG). Sistem ini mampu melakukan analisis real-time terhadap sinyal fisiologis yang diperoleh dari input video webcam, serta menyediakan visualisasi dan analisis pola pernapasan dan sinyal kardiovaskular secara simultan. Sinyal respirasi diambil dari gerakan bahu yang terdeteksi menggunakan model pose (pose landmark), sedangkan sinyal rPPG diambil dari wajah yang terdeteksi melalui face detection menggunakan bounding box. Setelah itu, sinyal-sinyal ini diproses dengan filter digital seperti bandpass filter untuk menghilangkan noise dan fokus pada frekuensi yang relevan. Terdapat dua aplikasi utama dalam proyek ini:

1. Melalui Jupyter Notebook untuk eksplorasi dan pemrosesan manual.
2. Aplikasi Web Interaktif menggunakan Streamlit yang memungkinkan pengaturan parameter filter secara real-time dan menampilkan sinyal mentah serta sinyal yang telah difilter.

Sistem ini tidak menggunakan alat tambahan seperti sensor, sehingga sepenuhnya berbasis kamera, dan bisa dijalankan di komputer dengan webcam. Proyek ini menggabungkan ilmu pengolahan sinyal digital, computer vision, dan visualisasi data real-time untuk membentuk sistem monitoring sederhana berbasis kamera.

## 2 Fitur Utama

Proyek ini memiliki beberapa fitur utama yang dirancang untuk mendukung proses pengambilan, pemrosesan dan visualisasi sinyal fisiologis secara real-time. Berikut ini adalah fitur yang digunakan :

- Pemrosesan Video Real-time: Mengambil input video dari webcam secara real-time. Frame video ini menjadi input utama untuk proses ekstraksi sinyal respirasi dan rPPG.
- Deteksi wajah untuk rPPG: Wajah pengguna dideteksi menggunakan metode face detection berbasis bounding box. Area wajah yang terdeteksi kemudian digunakan sebagai region of interest (ROI) untuk mengekstrak sinyal rPPG dari perubahan warna kulit menggunakan POS (Plane-Orthogonal-to-Skin) Function.
- Deteksi Pose untuk Respirasi: Program menggunakan model pose estimation dari Mediapipe untuk mendeteksi posisi bahu kiri dan kanan. Pergerakan vertikal (sumbu Y) dari kedua bahu tersebut digunakan sebagai dasar untuk membentuk sinyal respirasi.
- Pemrosesan dan Filtering Sinyal: Setelah sinyal respirasi dan rPPG berhasil diambil, dilakukan Bandpass filter untuk menghilangkan noise dan menjaga frekuensi penting dari masing-masing sinyal. Pada sinyal Respirasi frekuensi dibatasi sekitar 0.1 – 0.5 Hz sedangkan pada sinyal rPPG sekitar 0.7 – 2.5 Hz.

- Visualisasi Sinyal: Hasil ekstraksi sinyal divisualisasikan secara real-time menggunakan matplotlib untuk plotting dasar dalam notebook. Sedangkan streamlit digunakan sebagai visualisasi tampilan antarmuka web dengan Plotly untuk grafik interaktif.
- Antarmuka Pengguna Interaktif: Aplikasi web dengan tampilan intuitif dan pengaturan parameter filter secara live.

### 3 Teknologi

Teknologi yang digunakan pada proyek ini adalah :

- Bahasa Pemrograman: Python 3.10+  
Digunakan sebagai bahasa utama karena memiliki banyak pustaka (library) yang mendukung pemrosesan sinyal, analisis data, computer vision, dan visualisasi interaktif.
- Jupyter Notebook(.ipynb)  
Untuk eksperimen, pengujian proses sinyal tahap demi tahap, dan dokumentasi interaktif.
- Mediapipe  
Digunakan untuk mendeteksi pose tubuh manusia (khususnya landmark bahu) dalam ekstraksi sinyal respirasi.
- OpenCV (cv2)  
Digunakan untuk pengolahan video secara real-time dari webcam dan deteksi wajah (face detection).
- NumPy dan SciPy  
Untuk manipulasi array, filtering sinyal dengan bandpass filter, dan pemrosesan numerik lainnya.
- Matplotlib  
Digunakan untuk menampilkan sinyal respirasi dan rPPG dalam bentuk grafik secara statis maupun dinamis.
- Streamlit  
Digunakan untuk membuat aplikasi web interaktif yang menampilkan video dan grafik sinyal real-time secara sederhana dan cepat.
- Plotly  
Digunakan untuk visualisasi sinyal secara interaktif terutama pada versi Streamlit.
- Perangkat Keras: Webcam dan Laptop  
Pada proyek ini menggunakan Laptop dengan sistem operasi Windows 11 dan webcam bawaan dari device. Program ini juga dapat dijalankan pada sistem operasi lainnya seperti Linux atau MacOS.
- Local Python Environment (.venv)  
Disarankan menggunakan virtual environment untuk menjaga konsistensi dependensi.

### 4 Penjelasan Kode Program

Kode program pada project ini dibagi kedalam 2 bagian, yaitu kode program signal processing Manual menggunakan Jupyter Notebook dan kode program signal processing menggunakan UI Web-base.

## 4.1 Pemrosesan Sinyal menggunakan Jupyter Notebook

### Import Dependencies

Beberapa dependencies yang digunakan pada program ini yaitu cv2, mediapipe, numpy, matplotlib, dan scipy.signal

```
1 import cv2
2 import numpy as np
3 import mediapipe as mp
4 from mediapipe.tasks import python
5 from mediapipe.tasks.python import vision
6 import matplotlib.pyplot as plt
7 from scipy.signal import butter, filtfilt, find_peaks
```

Kode 1: Imports Dependencies

### Setup MediaPipe Models

MediaPipe membutuhkan model eksternal dalam faceDetection dan pose Landmark. Dalam proyek ini menggunakan models yang bersumber dari Github Teknik Informatika ITERA. [Source models](#)

```
1 # Face Detector
2 base_model = "models/blaze_face_short_range.tflite"
3 base_options = python.BaseOptions(model_asset_path=base_model)
4 FaceDetectorOptions = vision.FaceDetectorOptions
5 VisionRunningMode = vision.RunningMode
6 face_options = FaceDetectorOptions(
7     base_options=base_options,
8     running_mode=VisionRunningMode.IMAGE,
9 )
10 face_detector = vision.FaceDetector.create_from_options(face_options)
11
12 # Pose Landmarker
13 model_path = "models/pose_landmarker.task"
14 BaseOptions = mp.tasks.BaseOptions
15 PoseLandmarkerOptions = vision.PoseLandmarkerOptions
16
17 pose_options = PoseLandmarkerOptions(
18     base_options=BaseOptions(model_asset_path=model_path),
19     running_mode=VisionRunningMode.IMAGE,
20     num_poses=1,
21     min_pose_detection_confidence=0.5,
22     min_pose_presence_confidence=0.5,
23     min_tracking_confidence=0.5,
24     output_segmentation_masks=False
25 )
26 pose_landmarker = vision.PoseLandmarker.create_from_options(pose_options)
```

Kode 2: MediaPipe Models

### POS (Plane-Orthogonal-to-Skin) Function

RPPG (Remote Photoplethysmography) adalah metode untuk mengambil sinyal denyut di bagian tubuh menggunakan metode remote (jarak jauh), tanpa menyentuh langsung kulit. POS menjadi Metode rPPG yang mengambil sinyal denyut dari tubuh berdasarkan analisis perubahan warna yang dipancarkan kulit, yang kemudian diubah menjadi sinyal fisiologis.

```

1
2 def POS(signal, **kwargs):
3     eps = 10**-9
4     X = signal
5     e, c, f = X.shape
6     w = int(1.6 * kwargs['fps'])
7
8     P = np.array([[0, 1, -1], [-2, 1, 1]])
9     Q = np.stack([P for _ in range(e)], axis=0)
10
11     H = np.zeros((e, f))
12     for n in np.arange(w, f):
13         m = n - w + 1
14
15         Cn = X[:, :, m:(n+1)]
16         M = 1.0 / (np.mean(Cn, axis=2) + eps)
17         M = np.expand_dims(M, axis=2)
18         Cn = np.multiply(Cn, M)
19
20         S = np.dot(Q, Cn)
21         S = S[0, :, :, :]
22         S = np.swapaxes(S, 0, 1)
23
24         S1 = S[:, 0, :]
25         S2 = S[:, 1, :]
26         alpha = np.std(S1, axis=1) / (eps + np.std(S2, axis=1))
27         alpha = np.expand_dims(alpha, axis=1)
28         Hn = np.add(S1, alpha * S2)
29         Hnm = Hn - np.expand_dims(np.mean(Hn, axis=1), axis=1)
30
31         H[:, m:(n + 1)] = np.add(H[:, m:(n + 1)], Hnm)
32
33     return H

```

Kode 3: POS Fuction

## Filter Function

Membuat fungsi bandpass filter untuk memfilter atau menyaring noise di luar rentang utama. Sehingga nilai yang diambil hanya nilai sinyal yang berada di antara lowcut (batas frekuensi bawah) dan highcut (batas frekuensi atas). filtfilt dipilih karena tidak menyebabkan pergeseran fasa (zero phase distortion), yang penting dalam pemrosesan sinyal fisiologis seperti rPPG atau respirasi. Pada code ini, saya membatasi order filter hanya 3, karena sudah cukup tajam untuk menyaring noise dari luar, lebih stabil dan ringan secara komputasi.

```

1
2 def butter_bandpass(lowcut, highcut, fs, order=3):
3     nyq = 0.5 * fs # Nyquist frequency
4     low = lowcut / nyq
5     high = highcut / nyq
6     b, a = butter(order, [low, high], btype='band')
7     return b, a
8
9 def bandpass_filter(data, lowcut, highcut, fs, order=3):
10     b, a = butter_bandpass(lowcut, highcut, fs, order=order)
11     y = filtfilt(b, a, data)
12     return y

```

Kode 4: Filter Fuction

## Main Program

Dibawah ini merupakan program utama dari proyek ini. Fungsi ini menangkap video dari webcam selama batas waktu yang ditentukan saat inisialisasi parameter. Pada program ini, webcam akan menangkap video selama 60 detik pada kamera 15 fps (menyesuaikan dengan camera internal device). Sehingga jumlah frames menjadi 900 frames. Setelah webcam menyala, selanjutnya akan dilakukan pendeteksian wajah untuk memengekstraksi sinyal rPPG melalui mean RGB pada area wajah yang dibatasi oleh bounding box dan mendeteksi pose tubuh untuk mengekstraksi sinyal respirasi berdasarkan pergerakan vertikal bahu menggunakan landmark shoulder. Karena OpenCV menangkap gambar dalam bentuk BGR, maka perlu dilakukannya konversi ke bentuk RGB.

```

1
2 def run_rppg_respiration_analysis():
3     # Initialize parameters
4     fps = 15 # Frames per second
5     time_window = 60 # Time window in seconds
6     frames = time_window * fps # Total number of frames to capture
7     count = 0 # Frame counter
8
9     # Face detection parameters
10    margin_x = 10 # Margin for face bounding box
11    scaling_factor = 0.8 # Scaling factor for face bounding box
12
13    # Data storage
14    r_signal, g_signal, b_signal = [], [], [] # save RGB signals values
15    shoulder_y_positions = [] # save shoulder Y positions for respiration analysis
16
17    # Start webcam
18    cap = cv2.VideoCapture(0)
19
20    try:
21        while count < frames: #check if we have captured enough frames
22            ret, frame = cap.read() #read frame from webcam
23            if not ret:
24                print("Error: Failed to capture video") # exit loop if no frame is captured
25                break
26
27            # Convert to RGB for MediaPipe
28            rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
29            frame_height, frame_width, _ = rgb_frame.shape
30
31            # Create MediaPipe Image
32            mp_image = mp.Image(
33                image_format=mp.ImageFormat.SRGB,
34                data=rgb_frame
35            )
36
37            # Face Detection for rPPG
38            face_results = face_detector.detect(mp_image) # initialize face detection
39            face_detected = False # first assume no face detected
40
41            if face_results.detections:
42                for detection in face_results.detections:
43                    # Get bounding box
44                    bboxC = detection.bounding_box # bounding box coordinates
45                    x, y, w, h = bboxC.origin_x, bboxC.origin_y, bboxC.width, bboxC.height
46
47                    # Adjust bounding box
48                    new_x = int(x + margin_x)
49                    new_y = int(y)
50                    new_w = int(w * scaling_factor)

```

```

51         new_h = int(h * scaling_factor)
52
53         # Prevent out of bounds
54         new_x = max(0, min(new_x, frame_width - 1))
55         new_y = max(0, min(new_y, frame_height - 1))
56         new_w = min(new_w, frame_width - new_x)
57         new_h = min(new_h, frame_height - new_y)
58
59         # Extract face ROI
60         face_roi = rgb_frame[new_y:new_y+new_h, new_x:new_x+new_w]
61
62         if face_roi.size > 0:
63             # Calculate mean RGB values
64             mean_rgb = cv2.mean(face_roi)[:3]
65             r_signal.append(mean_rgb[0])
66             g_signal.append(mean_rgb[1])
67             b_signal.append(mean_rgb[2])
68             face_detected = True
69
70             # Draw face bounding box
71             cv2.rectangle(frame, (new_x, new_y),
72                           (new_x + new_w, new_y + new_h), (0, 255, 0), 2)
73         break
74
75     # If no face detected, append zeros to maintain alignment
76     if not face_detected:
77         r_signal.append(0)
78         g_signal.append(0)
79         b_signal.append(0)
80
81     # Pose Detection for Respiration
82     pose_result = pose_landmarker.detect(mp_image) # initialize pose detection
83     pose_detected = False # first assume no pose detected
84
85     if pose_result.pose_landmarks:
86         landmarks = pose_result.pose_landmarks[0] # Get first detected pose landmarks
87
88         # Get shoulder landmarks
89         left_shoulder = landmarks[11] # Left shoulder
90         right_shoulder = landmarks[12] # Right shoulder
91
92         # Calculate average shoulder Y position
93         avg_shoulder_y = (left_shoulder.y + right_shoulder.y) / 2 # Average Y position of
shoulders
94         shoulder_y_positions.append(avg_shoulder_y) # save shoulder Y position
95         pose_detected = True # pose detected
96
97         # Draw shoulder points and chest ROI
98         left_x_px = int(left_shoulder.x * frame_width)
99         left_y_px = int(left_shoulder.y * frame_height)
100        right_x_px = int(right_shoulder.x * frame_width)
101        right_y_px = int(right_shoulder.y * frame_height)
102
103        cv2.circle(frame, (left_x_px, left_y_px), 5, (255, 0, 0), -1)
104        cv2.circle(frame, (right_x_px, right_y_px), 5, (255, 0, 0), -1)
105
106        # Draw chest ROI
107        center_x = int((left_shoulder.x + right_shoulder.x) * frame_width / 2)
108        center_y = int((left_shoulder.y + right_shoulder.y) * frame_height / 2)
109        roi_left = max(0, center_x - 140)
110        roi_right = min(frame_width, center_x + 140)
111        roi_top = max(0, center_y - 80)

```

```

112         roi_bottom = min(frame_height, center_y)
113
114         cv2.rectangle(frame, (roi_left, roi_top),
115                       (roi_right, roi_bottom), (0, 0, 255), 2)
116
117         # If no pose detected, append zero to maintain alignment
118         if not pose_detected:
119             shoulder_y_positions.append(0)
120
121         # Display frame
122         cv2.putText(frame, f"Frame: {count}/{frames}", (10, 30),
123                    cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
124         cv2.putText(time_window, f"Time: {count / fps:.2f}s", (10, 60),
125                    cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
126         cv2.imshow('rPPG + Respiration Analysis', frame)
127
128         # exit if "q" is pressed
129         if cv2.waitKey(1) & 0xFF == ord('q'):
130             break
131
132         count += 1
133
134     finally:
135         #release webcam and close all window
136         cap.release()
137         cv2.destroyAllWindows()
138
139     return collected_signal
140     return r_signal, g_signal, b_signal, shoulder_y_positions

```

Kode 5: Main Program

## Signal Processing dan Analisis

Pada code ini akan melakukan analisis sinyal rPPG dari nilai warna RGB wajah untuk memperkirakan detak jantung (heart rate) serta menganalisis sinyal pernapasan dari gerakan vertikal bahu untuk memperkirakan laju pernapasan.

```

1
2 # Signal Processing and Analysis
3 def analyze_signals(r_signal, g_signal, b_signal, shoulder_y_positions, fps=15):
4     # Convert input list to numpy arrays
5     r_signal = np.array(r_signal)
6     g_signal = np.array(g_signal)
7     b_signal = np.array(b_signal)
8     shoulder_y_positions = np.array(shoulder_y_positions)
9
10    # Remove zero values (frames where detection failed)
11    valid_face_indices = (r_signal != 0) & (g_signal != 0) & (b_signal != 0)
12    valid_pose_indices = shoulder_y_positions != 0
13
14    results = {} #to store final output
15
16    # rPPG Analysis (Heart Rate)
17    if np.sum(valid_face_indices) > fps * 10: # se only if at least 10 seconds of valid face data
18        # Use POS method
19        rgb_signals = np.array([r_signal[valid_face_indices],
20                                g_signal[valid_face_indices],
21                                b_signal[valid_face_indices]])
22        rgb_signals = rgb_signals.reshape(1, 3, -1) # Shape: (1, 3, time)
23
24    # Apply POS algorithm to extract pulse signal

```

```

25     rppg_signal = POS(rgb_signals, fps=fps)
26     rppg_signal = rppg_signal.reshape(-1)
27
28     # Filter for heart rate (0.7-2.5 Hz)
29     if len(rppg_signal) > fps * 5: # Only continue if there's at least 5 seconds of data
30         filtered_rppg = bandpass_filter(rppg_signal, 0.7, 2.5, fps) # Find peaks in the
31         filtered signal (representing heart beats)
32         rppg_peaks, _ = find_peaks(filtered_rppg, distance=int(fps/4)) # Minimum distance
33         between
34         time_rppg = np.arange(len(filtered_rppg)) / fps
35
36     if len(rppg_peaks) > 1:
37         # Calculate heart rate (beats per minute)p
38         heart_rate = len(rppg_peaks) * 60 / time_rppg[-1]
39         results['rppg'] = {
40             'signal': filtered_rppg,
41             'peaks': rppg_peaks,
42             'time': time_rppg,
43             'heart_rate': heart_rate
44         }
45
46     # Respiration Analysis
47     if np.sum(valid_pose_indices) > fps * 10: # At least 10 seconds of data
48         valid_shoulder_y = shoulder_y_positions[valid_pose_indices] #
49
50         # Filter for respiration (0.1-0.5 Hz)
51         if len(valid_shoulder_y) > fps * 5: # At least 5 seconds of data
52             # Apply bandpass filter for respiration rate (0.1 - 0.5 Hz = 6 - 30 breaths per minute)
53             filtered_resp = bandpass_filter(valid_shoulder_y, 0.1, 0.5, fps) # Find peaks in the
54             filtered respiration signal (representing breaths)
55             resp_peaks, _ = find_peaks(filtered_resp, distance=int(fps/2)) # Minimum distance
56             between peaks
57             time_resp = np.arange(len(filtered_resp)) / fps
58
59         if len(resp_peaks) > 1:
60             # Calculate respiration rate (breaths per minute)
61             resp_rate = len(resp_peaks) * 60 / time_resp[-1]
62             results['respiration'] = {
63                 'signal': filtered_resp,
64                 'peaks': resp_peaks,
65                 'time': time_resp,
66                 'resp_rate': resp_rate
67             }
68
69     return results

```

Kode 6: Signal Processing and Analysis

## Visualisasi

Code ini digunakan untuk memvisualisasikan hasil analisis dari sinyal fisiologis. Visualisasi ini membantu mengevaluasi kualitas sinyal, melihat posisi puncak, dan interpretasi data fisiologis seperti detak jantung dan laju pernapasan.

```

1
2 def plot_results(results):
3     # Create a figure with 2 subplots vertically stacked
4     fig, axes = plt.subplots(2, 1, figsize=(15, 10))
5
6     # Plot rPPG
7     if 'rppg' in results:
8         rppg_data = results['rppg']

```



```

9
10     # Plot the filtered rPPG signal (blue line)
11     axes[0].plot(rppg_data['time'], rppg_data['signal'], 'b-', label='Filtered rPPG')
12     # Overlay the detected peaks (red dots)
13     axes[0].plot(rppg_data['time'][rppg_data['peaks']], rppg_data['signal'][rppg_data['peaks']
14 ], 'ro', label='Peaks')
15
16     # title, labels, legend, and grid
17     axes[0].set_title(f'rPPG Signal (Heart Rate: {rppg_data["heart_rate"]:.1f} BPM)')
18     axes[0].set_xlabel('Time (s)')
19     axes[0].set_ylabel('Amplitude')
20     axes[0].legend()
21     axes[0].grid(True)
22 else: # If no valid rPPG data is available
23     axes[0].set_title('No valid rPPG data')
24     axes[0].text(0.5, 0.5, 'Insufficient face detection data', ha='center', va='center',
25 transform=axes[0].transAxes)
26
27 # Plot Respiration
28 if 'respiration' in results:
29     resp_data = results['respiration']
30
31     # Plot the filtered respiration signal (green line)
32     axes[1].plot(resp_data['time'], resp_data['signal'], 'g-', label='Filtered Respiration')
33     # Overlay the detected peaks (red dots)
34     axes[1].plot(resp_data['time'][resp_data['peaks']], resp_data['signal'][resp_data['peaks']
35 ], 'ro', label='Peaks')
36
37     # Add title, labels, legend, and grid
38     axes[1].set_title(f'Respiration Signal (Rate: {resp_data["resp_rate"]:.1f} breaths/min)')
39     axes[1].set_xlabel('Time (s)')
40     axes[1].set_ylabel('Shoulder Y Position')
41     axes[1].legend()
42     axes[1].grid(True)
43 else: # If no valid respiration data is available
44     axes[1].set_title('No valid respiration data')
45     axes[1].text(0.5, 0.5, 'Insufficient pose detection data', ha='center', va='center',
46 transform=axes[1].transAxes)
47
48 plt.tight_layout()
49 plt.show()

```

Kode 7: Visualization

## Main Execution

Main execution block dalam Python berfungsi untuk menjalankan skrip hanya jika file ini dijalankan secara langsung, bukan ketika di-import oleh file lain.

```

1 if __name__ == "__main__":
2     print("Starting rPPG and Respiration Analysis...")
3     print("Press 'q' to quit early")
4
5     # Collect data
6     r_sig, g_sig, b_sig, shoulder_y = run_rppg_respiration_analysis()
7
8     print(f"Collected {len(r_sig)} frames of data")
9     print("Processing signals...")
10
11     # Analyze signals
12     results = analyze_signals(r_sig, g_sig, b_sig, shoulder_y)
13
14     # Display results

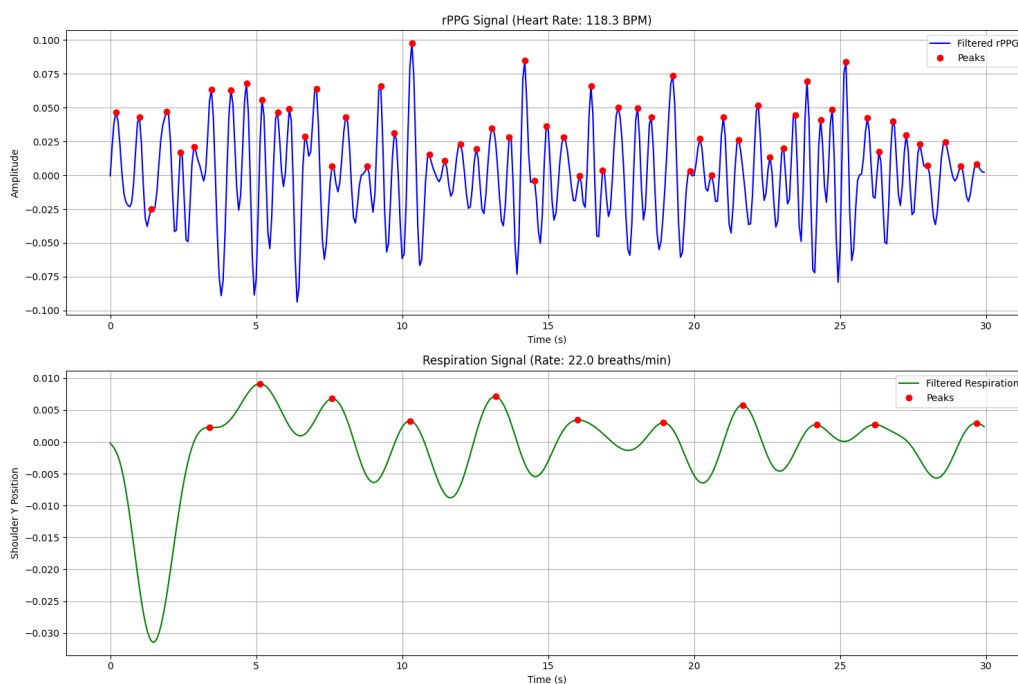
```

```

15 if 'rppg' in results:
16     print(f"Heart Rate: {results['rppg']['heart_rate']:.1f} BPM")
17 else:
18     print("Could not calculate heart rate - insufficient face detection")
19
20 if 'respiration' in results:
21     print(f"Respiration Rate: {results['respiration']['resp_rate']:.1f} breaths/min")
22 else:
23     print("Could not calculate respiration rate - insufficient pose detection")
24
25 # Plot results
26 plot_results(results)

```

Kode 8: main execution class



Gambar 1: Signal Visualization for 15fps and 30s

Gambar diatas adalah hasil visualisasi dari sinyal rPPG dan juga sinyal respirasi.

## 4.2 Pemrosesan Sinyal Web-based User Interface (app.py)

Pada program ini dijalankan pada file app.py menggunakan streamlit.

### Import Library

al pertama yang dilakukan adalah mengimport library yang dibutuhkan untuk membangun website ini.

```

1
2 import streamlit as st
3 import cv2
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from scipy import signal
7 from scipy.signal import find_peaks
8 import mediapipe as mp
9 import time
10 import plotly.graph_objects as go

```

```

11 from plotly.subplots import make_subplots
12
13 # Page configuration
14 st.set_page_config(
15     page_title="Simple rPPG Monitor",
16     page_icon="[ikon]",
17     layout="wide"
18 )

```

Kode 9: Imports Dependencies

## CSS Style

Agar tampilan website menjadi menarik, maka dilakukan styling melalui css.

## Inisialisasi MediaPipe Models

Inisialisasi MediaPipe model yang digunakan.

```

1 # Initialize MediaPipe
2 mp_face_detection = mp.solutions.face_detection
3 mp_pose = mp.solutions.pose
4
5 # Initialize session state
6 if 'recording' not in st.session_state:
7     st.session_state.recording = False
8 if 'rppg_data' not in st.session_state:
9     st.session_state.rppg_data = []
10 if 'resp_data' not in st.session_state:
11     st.session_state.resp_data = []
12 if 'timestamps' not in st.session_state:
13     st.session_state.timestamps = []
14 if 'capture_complete' not in st.session_state:
15     st.session_state.capture_complete = False

```

Kode 10: Inisialisasi MediaPipe Models

## Pengaturan fitur

Kode dibawah ini merupakan setingan dari setiap fitur-fitur di website, seperti pengaturan fps dan resolusi kamera, waktu capture/durasi time window, dan fitur lainnya.

```

1 # Main layout: Left column for settings, Right column for camera control and video
2 left_col, right_col = st.columns([1, 1])
3
4 with left_col:
5     # Camera Settings
6     st.markdown('<div class="settings-box">', unsafe_allow_html=True)
7     st.subheader("Camera Settings")
8     resolution = st.selectbox("Resolution", ["640x480", "1280x720", "1920x1080"], index=1)
9     fps = st.selectbox("FPS", [15, 30, 60], index=1)
10    st.markdown('</div>', unsafe_allow_html=True)
11
12    # Capture Settings
13    st.markdown('<div class="settings-box">', unsafe_allow_html=True)
14    st.subheader("Capture Settings")
15    capture_time = st.slider("Capture Duration (seconds)", 10, 120, 30)
16    st.markdown('</div>', unsafe_allow_html=True)
17
18 with right_col:
19    # Camera Control

```

```

20 st.markdown('<div class="control-box">', unsafe_allow_html=True)
21 st.subheader("Camera Control")
22
23 col_control1, col_control2, col_control3 = st.columns(3)
24
25 with col_control1:
26     start_capture = st.button("Start Capture", type="primary", use_container_width=True)
27
28 with col_control2:
29     stop_capture = st.button("Stop Capture", use_container_width=True)
30
31 with col_control3:
32     clear_data = st.button("Clear Data", use_container_width=True)
33
34 st.markdown('</div>', unsafe_allow_html=True)
35
36 # Status display
37 status_placeholder = st.empty()
38 progress_placeholder = st.empty()
39
40 # Video display container - always visible in right column
41 st.subheader("Camera Feed")
42 video_placeholder = st.empty()
43
44 # Default video container when not recording
45 if not st.session_state.recording:
46     st.markdown("""
47         <div class="video-container">
48             <div>
49                 <h3>Camera Ready</h3>
50                 <p>Click "Start Capture" to begin recording</p>
51                 <p>Camera feed will appear here</p>
52             </div>
53         </div>
54         """, unsafe_allow_html=True)

```

Kode 11: Pengaturan Fitur

## Membuat Functions

Membuat fungsi-fungsi yang akan digunakan dalam pemrosesan sinyal. Seperti fungsi ekstraksi sinyal, filter bandpass, menentukan dan menghitung titik puncak, serta fungsi visualisasi.

```

1 def extract_rppg_signal(face_roi):
2     """Simple green channel extraction for rPPG"""
3     if face_roi.size == 0:
4         return 0
5     return np.mean(face_roi[:, :, 1]) # Green channel
6
7 def extract_respiration_signal(pose_landmarks):
8     """Extract respiration from shoulder movement"""
9     if pose_landmarks is None:
10        return 0
11
12    try:
13        left_shoulder = pose_landmarks.landmark[mp_pose.PoseLandmark.LEFT_SHOULDER]
14        right_shoulder = pose_landmarks.landmark[mp_pose.PoseLandmark.RIGHT_SHOULDER]
15        # Use vertical movement of chest area
16        chest_y = (left_shoulder.y + right_shoulder.y) / 2
17        return chest_y
18    except:
19        return 0

```

```

20
21 def apply_bandpass_filter(data, lowcut, highcut, fs):
22     """Apply bandpass filter to signal"""
23     if len(data) < 4:
24         return data
25
26     nyquist = fs / 2
27     low = lowcut / nyquist
28     high = highcut / nyquist
29
30     # Ensure filter parameters are valid
31     low = max(0.01, min(low, 0.99))
32     high = max(low + 0.01, min(high, 0.99))
33
34     try:
35         b, a = signal.butter(4, [low, high], btype='band')
36         filtered_data = signal.filtfilt(b, a, data)
37         return filtered_data
38     except:
39         return data
40
41 def detect_peaks_and_calculate_rate(filtered_signal, fs):
42     """Detect peaks and calculate rate (BPM)"""
43     if len(filtered_signal) < fs:
44         return [], 0
45
46     # Find peaks
47     peaks, _ = find_peaks(filtered_signal, height=np.std(filtered_signal)*0.3, distance=fs//4)
48
49     if len(peaks) < 2:
50         return peaks, 0
51
52     # Calculate rate from peak intervals
53     peak_intervals = np.diff(peaks) / fs # Convert to seconds
54     avg_interval = np.mean(peak_intervals)
55     rate_bpm = 60 / avg_interval if avg_interval > 0 else 0
56
57     return peaks, rate_bpm
58
59 def create_signal_plot(timestamps, raw_rppg, raw_resp, filtered_rppg, filtered_resp, rppg_peaks,
60 resp_peaks, rppg_rate, resp_rate):
61     """Create visualization with both raw and filtered signals plus peaks"""
62     fig = make_subplots(
63         rows=4, cols=1,
64         subplot_titles=(
65             'Raw rPPG Signal (Captured)',
66             f'Filtered rPPG Signal - Heart Rate: {rppg_rate:.1f} BPM',
67             'Raw Respiration Signal (Captured)',
68             f'Filtered Respiration Signal - Breathing Rate: {resp_rate:.1f} BPM'
69         ),
70         vertical_spacing=0.08,
71         row_heights=[0.2, 0.3, 0.2, 0.3]
72     )
73     # Raw rPPG plot
74     fig.add_trace(
75         go.Scatter(x=timestamps, y=raw_rppg, mode='lines', name='Raw rPPG',
76             line=dict(color='lightcoral', width=1.5)),
77         row=1, col=1
78     )
79     # Filtered rPPG plot
80     fig.add_trace(

```

```

81     go.Scatter(x=timestamps, y=filtered_rppg, mode='lines', name='Filtered rPPG',
82               line=dict(color='red', width=2)),
83     row=2, col=1
84 )
85
86 # rPPG peaks
87 if len(rppg_peaks) > 0:
88     peak_times = [timestamps[i] for i in rppg_peaks if i < len(timestamps)]
89     peak_values = [filtered_rppg[i] for i in rppg_peaks if i < len(filtered_rppg)]
90     fig.add_trace(
91         go.Scatter(x=peak_times, y=peak_values, mode='markers', name='Heart Peaks',
92                   marker=dict(color='darkred', size=8, symbol='triangle-up')),
93         row=2, col=1
94     )
95
96 # Raw Respiration plot
97 fig.add_trace(
98     go.Scatter(x=timestamps, y=raw_resp, mode='lines', name='Raw Respiration',
99               line=dict(color='lightblue', width=1.5)),
100    row=3, col=1
101 )
102
103 # Filtered Respiration plot
104 fig.add_trace(
105     go.Scatter(x=timestamps, y=filtered_resp, mode='lines', name='Filtered Respiration',
106               line=dict(color='blue', width=2)),
107    row=4, col=1
108 )
109
110 # Respiration peaks
111 if len(resp_peaks) > 0:
112     peak_times = [timestamps[i] for i in resp_peaks if i < len(timestamps)]
113     peak_values = [filtered_resp[i] for i in resp_peaks if i < len(filtered_resp)]
114     fig.add_trace(
115         go.Scatter(x=peak_times, y=peak_values, mode='markers', name='Breath Peaks',
116                   marker=dict(color='darkblue', size=8, symbol='triangle-up')),
117         row=4, col=1
118     )
119
120 fig.update_layout(
121     height=800,
122     showlegend=True,
123     title_text="Raw vs Filtered Physiological Signals with Peak Detection"
124 )
125
126 fig.update_xaxes(title_text="Time (seconds)", row=4, col=1)
127 fig.update_yaxes(title_text="Amplitude", row=1, col=1)
128 fig.update_yaxes(title_text="Amplitude", row=2, col=1)
129 fig.update_yaxes(title_text="Amplitude", row=3, col=1)
130 fig.update_yaxes(title_text="Amplitude", row=4, col=1)
131
132 return fig
133

```

Kode 12: Membuat Functions

## Visualisasi with Additional Fitur Adjust Filter

```

1 # Results section (only shown after capture) - Full width below the main layout
2 if st.session_state.capture_complete and len(st.session_state.rppg_data) > 0:
3     st.markdown("---")

```

```

4     st.subheader("Signal Analysis Results")
5
6     # POST-CAPTURE FILTER ADJUSTMENT SECTION
7     st.markdown('<div class="filter-adjustment">', unsafe_allow_html=True)
8     st.subheader("Adjust Filter Parameters (Real-time)")
9     st.markdown("**Modify these values to see immediate changes in the filtered signal and peak  
detection:**")
10
11     filter_coll1, filter_coll2 = st.columns(2)
12
13     with filter_coll1:
14         st.markdown("**rPPG Filter Settings:**")
15         rppg_lowpass = st.slider("rPPG Lowpass (Hz)", 0.5, 4.0, 2.5, 0.1, key="post_rppg_low")
16         rppg_highpass = st.slider("rPPG Highpass (Hz)", 0.1, 2.0, 0.8, 0.1, key="post_rppg_high")
17
18     with filter_coll2:
19         st.markdown("**Respiration Filter Settings:**")
20         resp_lowpass = st.slider("Resp Lowpass (Hz)", 0.1, 1.0, 0.5, 0.05, key="post_resp_low")
21         resp_highpass = st.slider("Resp Highpass (Hz)", 0.05, 0.5, 0.1, 0.05, key="post_resp_high")
22
23     st.markdown('</div>', unsafe_allow_html=True)
24
25     # Process signals with current filter settings
26     rppg_array = np.array(st.session_state.rppg_data)
27     resp_array = np.array(st.session_state.resp_data)
28     time_array = np.array(st.session_state.timestamps)
29
30     # Apply filters with current settings
31     rppg_filtered = apply_bandpass_filter(rppg_array, rppg_highpass, rppg_lowpass, fps)
32     resp_filtered = apply_bandpass_filter(resp_array, resp_highpass, resp_lowpass, fps)
33
34     # Detect peaks and calculate rates
35     rppg_peaks, heart_rate = detect_peaks_and_calculate_rate(rppg_filtered, fps)
36     resp_peaks, breathing_rate = detect_peaks_and_calculate_rate(resp_filtered, fps)
37
38     # Display results with current filter settings
39     result_coll1, result_coll2, result_coll3, result_coll4 = st.columns(4)
40
41     with result_coll1:
42         st.metric("Heart Rate", f"{heart_rate:.1f} BPM", delta=None)
43         st.metric("Heart Peaks", len(rppg_peaks))
44
45     with result_coll2:
46         st.metric("Breathing Rate", f"{breathing_rate:.1f} BPM", delta=None)
47         st.metric("Breath Peaks", len(resp_peaks))
48
49     with result_coll3:
50         st.metric("Capture Duration", f"{len(st.session_state.timestamps)/fps:.1f}s")
51         st.metric("Data Points", len(st.session_state.rppg_data))
52
53     with result_coll4:
54         st.metric("rPPG Filter", f"{rppg_highpass:.1f}-{rppg_lowpass:.1f} Hz")
55         st.metric("Resp Filter", f"{resp_highpass:.2f}-{resp_lowpass:.2f} Hz")
56
57     # Create and display plot with both raw and filtered signals
58     signal_plot = create_signal_plot(
59         time_array, rppg_array, resp_array, rppg_filtered, resp_filtered,
60         rppg_peaks, resp_peaks, heart_rate, breathing_rate
61     )
62     st.plotly_chart(signal_plot, use_container_width=True)
63
64     # Show filter effect info

```

```

65 st.info("**Interactive Filtering**": Adjust the filter sliders above to see real-time changes in
    the processed signals and detected peaks. The raw signals remain unchanged, showing you the
    original captured data.")
66
67 # Raw data option
68 with st.expander("View Raw Data Details"):
69     col_raw1, col_raw2 = st.columns(2)
70     with col_raw1:
71         st.subheader("Raw rPPG Data")
72         st.line_chart(rppg_array)
73         st.write(f"Raw rPPG Stats:")
74         st.write(f"- Mean: {np.mean(rppg_array):.3f}")
75         st.write(f"- Std: {np.std(rppg_array):.3f}")
76         st.write(f"- Range: {np.max(rppg_array) - np.min(rppg_array):.3f}")
77
78     with col_raw2:
79         st.subheader("Raw Respiration Data")
80         st.line_chart(resp_array)
81         st.write(f"Raw Respiration Stats:")
82         st.write(f"- Mean: {np.mean(resp_array):.3f}")
83         st.write(f"- Std: {np.std(resp_array):.3f}")
84         st.write(f"- Range: {np.max(resp_array) - np.min(resp_array):.3f}")

```

Kode 13: Visualisasi and adjust filter

## Main Execution

```

1 # Main capture loop
2 if st.session_state.recording:
3     # Parse resolution
4     width, height = map(int, resolution.split('x'))
5
6     # Initialize camera
7     cap = cv2.VideoCapture(0)
8     cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
9     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
10    cap.set(cv2.CAP_PROP_FPS, fps)
11
12    # Initialize MediaPipe
13    with mp_face_detection.FaceDetection(min_detection_confidence=0.7) as face_detection, \
14        mp_pose.Pose(min_detection_confidence=0.5) as pose:
15
16        start_time = time.time()
17        frame_count = 0
18
19        status_placeholder.info(f"Recording for {capture_time} seconds...")
20        progress_bar = progress_placeholder.progress(0)
21
22        while st.session_state.recording:
23            ret, frame = cap.read()
24            if not ret:
25                status_placeholder.error("Failed to access webcam")
26                break
27
28            current_time = time.time() - start_time
29
30            # Check if capture time reached
31            if current_time >= capture_time:
32                st.session_state.recording = False
33                st.session_state.capture_complete = True
34                break

```



```

35
36     # Update progress
37     progress = min(current_time / capture_time, 1.0)
38     progress_bar.progress(progress)
39
40     # Convert to RGB
41     frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
42
43     # Face detection
44     face_results = face_detection.process(frame_rgb)
45     pose_results = pose.process(frame_rgb)
46
47     # Draw annotations
48     annotated_frame = frame_rgb.copy()
49
50     rppg_signal = 0
51     resp_signal = 0
52
53     # Process face for rPPG
54     if face_results.detections:
55         for detection in face_results.detections:
56             bbox = detection.location_data.relative_bounding_box
57             h, w, _ = frame_rgb.shape
58
59             x = int(bbox.xmin * w)
60             y = int(bbox.ymin * h)
61             width_box = int(bbox.width * w)
62             height_box = int(bbox.height * h)
63
64             # Draw face bounding box
65             cv2.rectangle(annotated_frame, (x, y), (x + width_box, y + height_box), (0,
255, 0), 2)
66             cv2.putText(annotated_frame, "rPPG ROI", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 255, 0), 2)
67
68             # Extract signal
69             face_roi = frame_rgb[y:y+height_box, x:x+width_box]
70             rppg_signal = extract_rppg_signal(face_roi)
71
72     # Process pose for respiration
73     if pose_results.pose_landmarks:
74         # Draw pose landmarks (simplified)
75         landmarks = pose_results.pose_landmarks.landmark
76         h, w, _ = frame_rgb.shape
77
78         # Draw shoulder points
79         left_shoulder = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER]
80         right_shoulder = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER]
81
82         cv2.circle(annotated_frame, (int(left_shoulder.x * w), int(left_shoulder.y * h)),
8, (255, 0, 0), -1)
83         cv2.circle(annotated_frame, (int(right_shoulder.x * w), int(right_shoulder.y * h)),
8, (255, 0, 0), -1)
84         cv2.putText(annotated_frame, "Resp ROI", (int(left_shoulder.x * w), int(
left_shoulder.y * h)-20),
85                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
86
87         resp_signal = extract_respiration_signal(pose_results.pose_landmarks)
88
89     # Store data
90     st.session_state.rppg_data.append(rppg_signal)
91     st.session_state.resp_data.append(resp_signal)

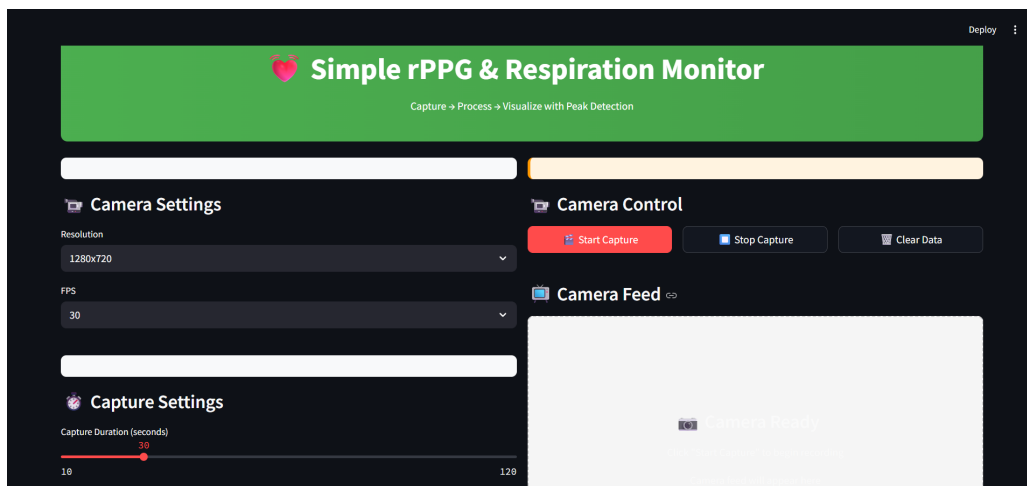
```

```

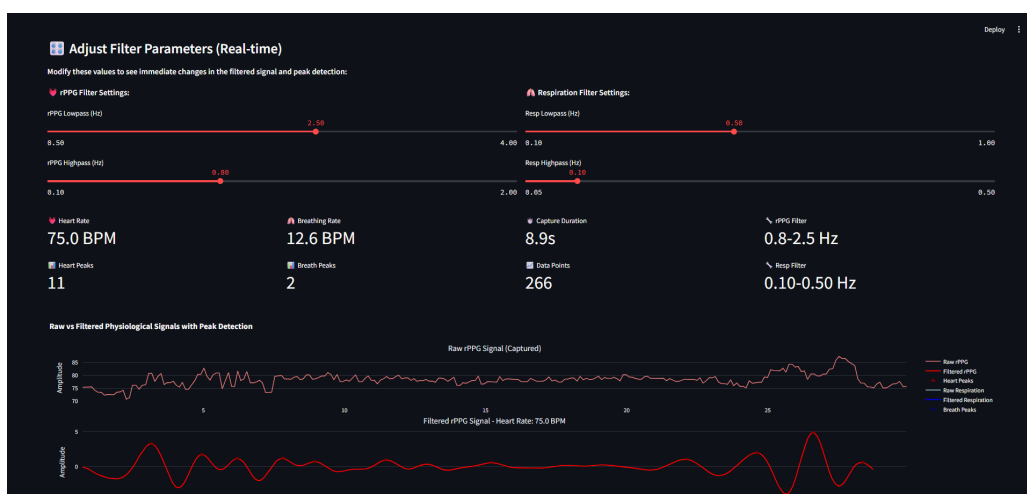
92     st.session_state.timestamps.append(current_time)
93
94     # Display frame in the right column
95     video_placeholder.image(annotated_frame, channels="RGB", use_column_width=True)
96
97     frame_count += 1
98     time.sleep(1/fps)
99
100 cap.release()
101 progress_placeholder.empty()
102
103 if st.session_state.capture_complete:
104     status_placeholder.success("Capture completed! Now you can adjust filter parameters below
105     to see real-time changes.")
106     st.rerun()

```

Kode 14: Main Execution



Gambar 2: Website UI



Gambar 3: Adjustable filter Signal Visualization

## 5 Cara Menjalankan Program

Ikuti langkah-langkah berikut untuk menjalankan proyek ini di komputer lokal Anda:

### 5.1 Clone repository

```
1 git clone https://github.com/dindajoycehana/TugasBesar_DSP_122140048.git
2 cd TugasBesar_DSP_122140048
3
4
5
```

### 5.2 Buat virtual environment (disarankan)

Jalankan perintah ini melalui terminal di VSCode. Pastikan Anda berada di folder yang tepat. Pada proyek ini, saya menggunakan virtual environment uv dan Python versi 3.10.

### 5.3 Install uv:

```
1 pip install uv
2
3
```

### 5.4 Buat virtual environment

```
1 uv venv --python=python3.10
2
3
```

### 5.5 Aktifkan virtual environment:

- Pada Windows:

```
1 venv\Scripts\activate
2
3
```

- Pada macOS/Linux:

```
1 source venv/bin/activate
2
3
```

### 5.6 Install dependencies yang dibutuhkan:

```
1 uv pip install -r requirements.txt
2
3
```

### 5.7 Menjalankan Program

Anda dapat menjalankan proyek ini dengan dua metode berbeda sesuai kebutuhan:

## Menggunakan Jupyter Notebook (*signal\_processing.ipynb*)

- Pastikan Anda sudah menginstall Jupyter:

```
1
2     uv pip install notebook
3
```

- Jalankan Jupyter Notebook:

```
1
2     jupyter notebook
3
```

- Buka file `signal_processing.ipynb` dan jalankan cell-cell secara berurutan.

## Menggunakan Aplikasi Streamlit(*app.py*)

- Pastikan Anda sudah menginstall Streamlit dari `requirements.txt` atau jalankan perintah berikut di terminal:

```
1
2     uv pip install streamlit plotly
3
```

- Jalankan aplikasi Streamlit:

```
1
2     streamlit run app.py
3
```

- Browser Anda akan terbuka secara otomatis di alamat: `http://localhost:8501`

## LAMPIRAN

Github Repository : [Dinda Repository](#)